



UNIVERSITÀ DEGLI STUDI DI NAPOLI
FEDERICO II

*DOCUMENTO DI SPECIFICA DEI REQUISITI SOFTWARE
"BUGBOARD26"*

CORSO DI INGEGNERIA DEL SOFTWARE

Gabriele Letizia - N86005267

Fabio Iannicelli - N86005383

ANNO ACCADEMICO 2025/26

Indice

1	Introduzione	3
1.1	Scopo	3
1.2	Obiettivi	3
1.3	Definizioni, acronimi e abbreviazioni	4
1.3.1	Definizioni	4
2	Modellazione tramite Use Case Diagram	8
3	Caratterizzazione degli Utenti	9
4	Requisiti Specifici	10
4.1	Requisiti Funzionali	10
4.2	Requisiti non-funzionali e di sistema	11
5	Specifica dei Casi d'Uso	13
5.1	UC1: Segnalazione di una Issue	13
5.2	UC2: Archiviazione di un Bug	14
5.3	Mock-Up UC1: Segnalazione di una Issue	15
5.4	Mock-Up UC2: Archiviazione di un Bug	15
6	Architettura Proposta	16
6.1	Back-end (Server)	16
6.2	Front-end (Client)	16
6.3	Criteri di Design e Astrazione	16
7	Scelte Tecnologiche	17
7.1	Tecnologie Back-end	17
7.2	Tecnologie Front-end	17
7.3	Strumenti di Sviluppo e Modellazione	18

1 Introduzione

1.1 Scopo

Questo documento ha lo scopo di definire le specifiche dei requisiti per una piattaforma dedicata alla gestione collaborativa di issue in progetti software. Il documento si rivolge a tutti gli utenti del sistema, illustrandone le relative specifiche software e i vincoli.

1.2 Obiettivi

Il prodotto software che si intende realizzare prende il nome di “BugBoard26”. Il sistema dovrà offrire le seguenti funzionalità:

- 1) Gestire l'autenticazione degli utenti, consentendo a ciascuno di accedere alle sole funzionalità che gli competono.
- 2) Consentire agli amministratori di creare nuove utenze, potendone assegnare il ruolo.
- 3) Permettere utenti interni e amministratori di segnalare nuove issue.
- 4) Fornire una vista riepilogativa delle issue, includendo la possibilità di filtrare o ordinare i risultati.
- 5) Fornire agli amministratori una dashboard contenente informazioni aggregate sui bug.
- 6) Consentire agli amministratori di archiviare i bug.
- 7) Fornire una sezione dove è possibile visualizzare tutti i bug archiviati.
- 8) Permettere agli amministratori di contrassegnare un bug come duplicato di un altro, comportandone la chiusura immediata.
- 9) Consentire visualizzazione dei bug da parte di utenti esterni con appositi account creati dagli amministratori.

Mentre non è prevista l'implementazione delle seguenti funzionalità:

- 1) Un sistema automatico di notifiche.
- 2) L'Esportazione dei dati, come ad esempio dell'elenco dei bug.
- 3) Sistema di messaggistica tra utenti.
- 4) Creazione di ulteriori ruoli rispetto ai tre previsti ("Utente", "Amministratore" e "Utente esterno").
- 5) Un sistema di registrazione alla piattaforma da parte degli utenti.

BugBoard26 ha come obiettivo principale quello di affiancare team di sviluppo, fungendo da piattaforma di segnalazione di problemi relativi a un progetto e di monitorarne lo stato, ottimizzando così la collaborazione operativa tra i membri del team.

1.3 Definizioni, acronimi e abbreviazioni

1.3.1 Definizioni

- **Issue** – Indica un elemento che rappresenta qualcosa da risolvere o da gestire all'interno di un progetto.
- **Amministratore** – Indica un ruolo "speciale" di utente, che ha uno scopo di gestione, come creare nuove utenze e la gestione degli account.
- **Utente** – Indica un'utenza "normale", che può usufruire di tutte le funzionalità che gli competono.
- **Bug** – È una issue per segnalare un malfunzionamento.
- **Question** – È una issue usata per richieste di chiarimenti.
- **Feature** – È una issue per indicare la richiesta o il suggerimento di nuove funzionalità.
- **Documentation** – È una issue per segnalare problemi relativi alla documentazione
- **Team** – È un gruppo di persone che collaborano insieme per portare a termine attività o progetti.
- **Dashboard** – È un'interfaccia che mostra in modo chiaro e riassuntivo le principali informazioni sui bug del sistema.
- **Utenti esterni** – Persone che non fanno parte del team interno di sviluppo o gestione del progetto, ma che hanno interesse a seguirne l'andamento.
- **BugBoard26** - È il nome usato per identificare il software.
- **Software** – Insieme di programmi e istruzioni che permettono di svolgere determinate funzioni o compiti.
- **Modalità read-only** - Modalità di accesso che permette solo visualizzazione dei contenuti per utenti esterni, in questo caso bug, senza possibilità di modificarli.
- **Stato della Issue** – Indica la fase del ciclo di vita di una issue. Una issue può trovarsi negli stati "Todo", "Assegnato", "Risolto" e "Archiviato".
- **E-mail** - Strumento informatico per l'invio della posta elettronica.
- **Password** – Insieme di caratteri alfanumerici che abilitano l'accesso ad aree riservate a determinati utenti.
- **Interfaccia utente** – È la parte di software con cui l'utente interagisce direttamente. Include tutti gli elementi visivi e di controllo, come menù, pulsanti, finestre, icone e moduli, che permettono di usare il programma in modo semplice ed efficace.

- **Autenticazione** - È il processo tramite il quale un sistema verifica l'identità di un utente prima di permettergli l'accesso. Serve a garantire che solo persone autorizzate possano entrare in un'applicazione, un sito o un servizio.
- **Use Case Diagram** - È un tipo di diagramma UML che serve a rappresentare graficamente le funzionalità principali di un sistema viste dal punto di vista dell'utente. Mostra chi (gli attori) interagisce con il sistema e quali azioni (i casi d'uso) può compiere.
- **Autorizzazione** - Processo mediante il quale il sistema determina quali risorse e funzionalità possono essere utilizzate da un utente autenticato, sulla base dei permessi associati al suo ruolo.
- **UML (Unified Modeling Language)** - Linguaggio di modellazione standard usato per rappresentare graficamente la struttura, il comportamento e le interazioni di un sistema software.
- **Todo** - Indica un'attività ancora da completare o un promemoria, in questo caso, la gestione di una issue.
- **Back-end** - È la componente del sistema responsabile della gestione della logica e dei dati. Deve essere completamente indipendente dal front-end.
- **Front-end** - È la componente che rappresenta l'interfaccia utente dell'applicazione. Si appoggia ai servizi del back-end esclusivamente tramite chiamate di rete e deve poter essere eseguito separatamente da esso.
- **Template di A. Cockburn** - È una tabella utilizzata per fornire la descrizione testuale dettagliata dei casi d'uso all'interno del documento di specifica.
- **Tool di CASE** - Strumenti software, come Visual Paradigm, che vengono utilizzati per lo svolgimento delle attività di progettazione e modellazione UML.
- **API REST (Application Programming Interface)** - Sono le interfacce di programmazione accessibili via rete esposte dal back-end, attraverso le quali il front-end comunica per scambiare dati.
- **Mock-up** - È un prototipo visuale utilizzato nella documentazione per rappresentare graficamente la progettazione delle interfacce utente relative ai casi d'uso.
- **MBaaS (Mobile Backend as a Service)** - Modello di cloud computing in cui l'infrastruttura e la logica di back-end vengono fornite e gestite interamente da terze parti.
- **Docker** - Tecnologia che permette di creare dei container, cioè impacchettare un'applicazione e tutte le sue dipendenze, garantendo che il software funzioni in modo identico in qualsiasi ambiente.
- **Client-Server** - Architettura di rete in cui un componente (Client) richiede servizi o delle risorse, e un'altra componente (Server) li elabora e li fornisce.

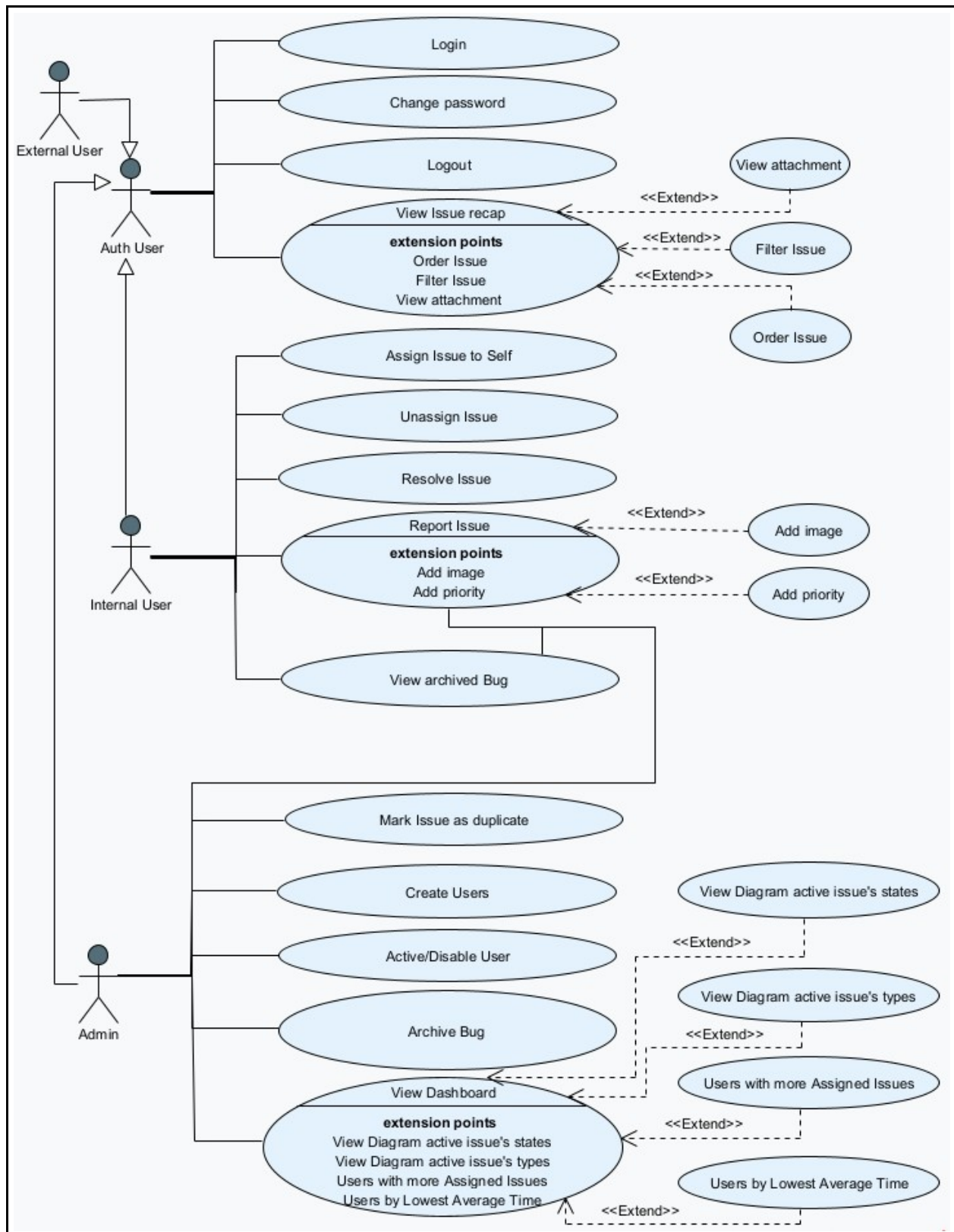
- **Framework** - Struttura di supporto su cui un software può essere progettato e realizzato, fornisce funzionalità preimpostate (ad esempio Spring Boot).
- **Rollback** - Operazione che consente di riportare una transazione o un sistema a uno stato precedente, annullando le modifiche effettuate nell'ambito dell'operazione.
- **Sistema di Versioning** - Strumento software (come ad esempio Git) che traccia, storicizza e gestisce tutte le modifiche apportate al codice sorgente nel corso del tempo da un team di sviluppo.
- **JWT (JSON Web Token)** - Standard utilizzato per rappresentare informazioni relative all'identità e alle autorizzazioni di un utente sotto forma di token, consentendone la trasmissione tra Client e Server.
- **SonarQube** - Strumento per l'ispezione della qualità del codice, utilizzata per eseguire analisi e rilevare automaticamente bug, cattive pratiche di programmazione, vulnerabilità e codice duplicato.
- **Dependency Injection** - Principio in cui le dipendenze di un oggetto (le altre classi di cui ha bisogno per funzionare) gli vengono fornite dall'esterno, favorendo un forte disaccoppiamento del codice.
- **Object-Oriented (OO)** - Paradigma di programmazione basato sul concetto di oggetti, cioè strutture che incapsulano al loro interno informazioni (attributi) e i comportamenti (metodi).
- **SOLID** - Acronimo che raggruppa cinque principi fondamentali (S: Single responsibility, O: Open-Closed, L: Liskov substitution, I: Interface segregation, D: Dependency inversion) del paradigma OO, volti a rendere il software più flessibile alle modifiche e più manutenibile.
- **Java** - Linguaggio di programmazione orientato agli oggetti, ad alto livello e fortemente tipizzato. È stato progettato per essere indipendente dalla piattaforma hardware di esecuzione.
- **JavaFx** - Piattaforma e libreria grafica utilizzata per la creazione e progettazione di interfacce utente per applicazioni desktop in ambiente Java.
- **PostgreSQL** - Sistema di gestione di database relazionali, utilizzato per la memorizzazione e la gestione persistente dei dati del sistema.
- **Architettura a Livelli** - Architettura in cui il software è suddiviso in strati separati e indipendenti, ognuno con una responsabilità specifica.
- **Model** - Strato dell'architettura che rappresenta le entità fondamentali del sistema.
- **Repository** - Componente dell'architettura responsabile dell'accesso ai dati persistenti e dell'interazione con il database, isolando tale responsabilità dalla logica applicativa.

Documento di specifica dei requisiti software

- **Service** - Componente dell'architettura intermedio che gestisce l'intera logica di business e le regole applicative, operando tra Controller e i Repository.
- **Controller** - Componente dell'architettura che funge da coordinatore. Espone le API ricevendo le richieste di rete e restituendo le risposte;
- **FXML** - Linguaggio di markup basato su XML utilizzato in JavaFx per definire la struttura dell'interfaccia grafica (View) in modo separato dal codice logico (Controller).
- **Linguaggio di Markup (o di marcatura)** - Sistema di regole e convenzioni per definire la struttura, la formattazione e la disposizione degli elementi in un documento o in un'interfaccia (es. XML, HTML, FXML).
- **Classi Helper** - Classi di supporto progettate per contenere metodi e funzioni generiche e riutilizzabili da diverse parti dell'applicazione, al fine di evitare duplicazioni di codice.
- **Stateless** - Architettura nella quale il server non mantiene lo stato della sessione del client tra richieste successive; ogni richiesta contiene le informazioni necessarie per essere elaborata.

Spring Security Spring Data JPA Hibernate/JPA REST JSON HTTP JWT API ruolo
autenticazione autorizzazione issue state transazione

2 Modellazione tramite Use Case Diagram



3 Caratterizzazione degli Utenti

Gli utenti di BugBoard26 devono avere una conoscenza base dell'uso del computer. In base al ruolo, ogni utente può accedere esclusivamente a determinate funzionalità ed ai dati di loro competenza. Gli utenti di BugBoard26 sono:

- **Amministratore** - È un utente speciale che ha accesso completo a tutte le funzionalità del sistema e può visualizzare dati aggregati e statistici attraverso una Dashboard di controllo. Ha dunque in particolar modo un ruolo di gestione.
- **Utente interno** - È un classico utente che rappresenta un membro del team di sviluppo che accede al sistema tramite credenziali personali (email e password). Non dispone di permessi amministrativi e può operare solo sulle issue di sua competenza.
- **Utente esterno** - È un soggetto non appartenente al team interno di sviluppo, ma interessato a consultare lo stato delle issue (ad esempio stakeholder o clienti). Account di questo tipo sono creati dagli amministratori e servono per garantire trasparenza e monitoraggio verso soggetti esterni all'organizzazione.

4 Requisiti Specifici

4.1 Requisiti Funzionali

BugBoard26 prevede le seguenti funzionalità:

- **Autenticazione:** All'avvio di BugBoard26, tutti gli utenti devono autenticarsi, tramite una coppia di e-mail e una password, sarà poi il sistema a verificare che questa coppia sia valida, dando all'utente l'accesso alle funzionalità che gli competono.
- **Cambio credenziali:** Una volta autenticato, ogni utente deve avere la possibilità di aggiornare la password del proprio account.
- **Logout:** Ogni utente autenticato deve avere la possibilità di disconnettersi dal sistema.
- **Vista Riepilogativa Delle Issue:** Il sistema deve dare la possibilità a tutti gli utenti di poter visualizzare una vista riepilogativa delle issue, con la possibilità di filtrare o ordinare i risultati.
- **Visualizzazione degli allegati:** Il sistema deve dare la possibilità a tutti gli utenti di poter visualizzare un'immagine allegata ad una issue segnalata.
- **Prendi in carico:** Un utente interno può selezionare e assegnarsi una issue su cui lavorare.
- **Rilascia issue:** Un utente interno può selezionare una issue, tra quelle a lui assegnate, e scegliere di abbandonarla.
- **Segna come risolta:** Un utente interno può selezionare una issue, tra quelle a lui assegnate, e impostare lo stato a risolto.
- **Segnalazione di una Issue:** Un utente interno può segnalare una nuova issue andando a specificare almeno un titolo, una descrizione e il tipo; quest'ultimo può anche voler specificare una priorità ed allegare un'immagine. Le issue possono essere di diverso tipo: Question, bug, Documentation, e Feature. Appena segnalata una issue è nello stato "todo".
- **Visualizzazione dei Issue archiviate:** Un utente interno può visualizzare la lista delle issue archiviate, con la possibilità di cercare o filtrare in maniera specifica.
- **Contrassegnazione di un issue come duplicata:** Un amministratore può contrassegnare un bug come duplicato di un altro, in modo tale da non riempire la lista con dei duplicati; la contrassegnazione comporta la chiusura immediata del bug duplicato.
- **Creazione di un utente:** Un amministratore può scegliere di creare una nuova utenza, andando a specificare una e-mail, una password e un ruolo (utente interno/esterno o amministratore), il sistema poi verificherà che quella e-mail non sia già registrata.
- **Attiva/Disabilita utente:** Un amministratore deve avere la possibilità di attivare o disattivare un utente interno/esterno del sistema.

- **Archiviazione di un Bug:** Un amministratore può visualizzare la lista dei bug presenti, scegliere se archiviare un bug, in modo che non appaia più nelle liste principali ma resti comunque accessibile da una pagina dedicata.
- **Visualizzazione utenti del sistema:** Un amministratore può visualizzare una tabella contenente una lista di tutte le utenze del sistema. In particolar modo potrà visualizzarne email, ruolo e lo stato dell'account (attivo/disattivato).
- **Visualizzazione Dashboard:** Il sistema deve dare la possibilità ad un amministratore di visualizzare una dashboard riepilogativa. Quest'ultima deve consentire all'amministratore di visualizzare informazioni aggregate relative alle issue attive, tra cui la distribuzione per stato e tipologia, il numero di issue assegnate agli utenti e il tempo medio di risoluzione, con possibilità di individuare gli utenti caratterizzati dai minori tempi medi di risoluzione.

4.2 Requisiti non-funzionali e di sistema

BugBoard26 deve rispettare i seguenti vincoli architetturali e prestazionali:

- **Prestazioni:** Il sistema deve garantire tempi di risposta contenuti per le operazioni principali, evitando ritardi percepibili dall'utente in condizioni operative ordinarie. La generazione della dashboard deve mantenere tempi di risposta adeguati anche in presenza di un elevato numero di issue.
- **Affidabilità:** Il sistema deve garantire integrità e consistenza dei dati anche in caso di crash o interruzioni. Devono essere previsti meccanismi di rollback e gestione eccezioni a livello di rete e applicazione.
- **Sicurezza:** Il sistema deve garantire l'autenticazione degli utenti e l'autorizzazione delle operazioni sulla base del ruolo associato all'account, impedendo l'accesso alle funzionalità non consentite.
- **Usabilità:** L'interfaccia utente deve essere intuitiva e coerente, le funzionalità principali devono essere raggiungibili con meno interazioni possibili. L'applicazione deve essere facilmente utilizzabile anche da utenti esterni in modalità "readonly".
- **Paradigma di Programmazione:** L'implementazione deve utilizzare obbligatoriamente un linguaggio di programmazione Object-Oriented.
- **Architettura Distribuita:** Il sistema deve essere implementato prevedendo due macro-componenti indipendenti: un Back-end e un Front-end.
- **Indipendenza del Front-end:** Il Front-end deve comunicare con il Back-end esclusivamente attraverso la rete, sfruttando le API esposte, e deve poter essere eseguito separatamente.
- **Gestione Centralizzata (Back-end):** Il Back-end gestisce centralmente la logica applicativa e la persistenza dei dati del sistema.

- **Manutenibilità:** Il codice sorgente deve essere commentato, modulare e aderente ai principi SOLID. Devono essere utilizzati strumenti di versioning (es. GitHub) per il tracciamento delle modifiche e per la collaborazione. È richiesto l'impiego di un CASE tool (es. Visual Paradigm) per favorire la documentazione e l'evoluzione del progetto.
- **Testabilità:** Il sistema deve consentire l'esecuzione di test automatici di unità e deve supportare l'analisi della qualità del codice tramite strumenti come ad esempio SonarQube.

5 Specifica dei Casi d'Uso

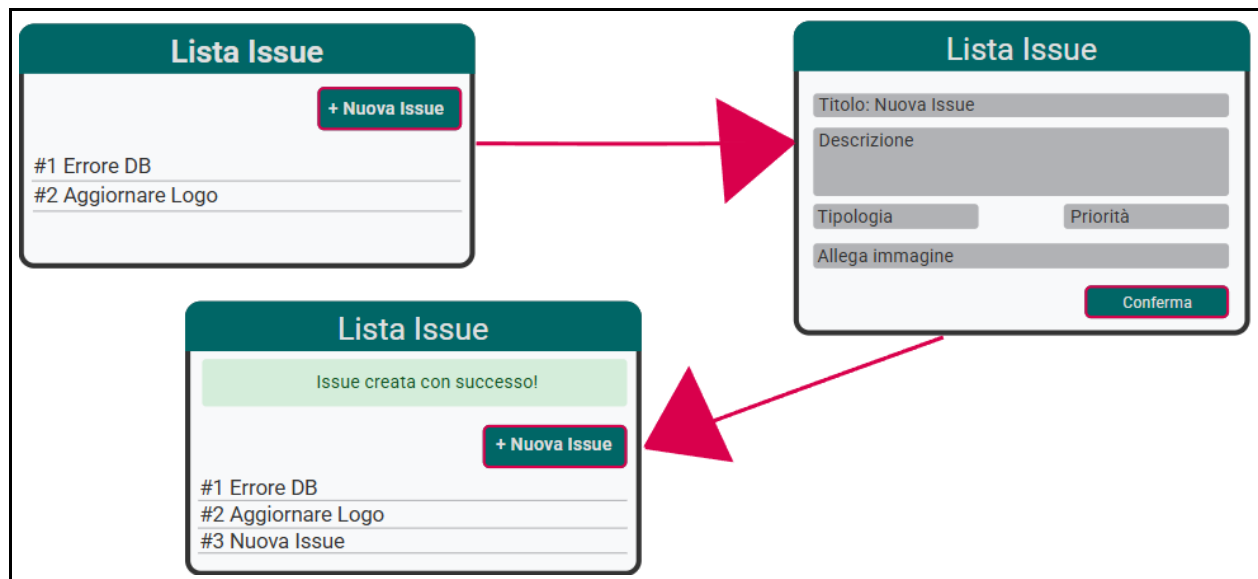
5.1 UC1: Segnalazione di una Issue

USE CASE #1	Segnalazione di una Issue		
Goal in Context	Consentire a un utente/amministrato di segnalare una issue.		
Preconditions	L'utente deve essere autenticato nel sistema.		
Success End Condition	La issue viene salvata nel sistema nello stato "todo".		
Failed End Condition	La issue non viene creata (es. campi obbligatori mancanti).		
Primary Actor	Utente interno o Amministratore		
Trigger	L'utente clicca sul pulsante "Nuova Issue".		
Main Scenario	Step n.	Actor 1 (Utente)	System
	1		Mostra il form di inserimento per una nuova issue
	2	Inserisce titolo e descrizione	
	3	Specifica tipo e può anche specificare priorità e allegare immagine	
	4	Conferma la creazione	
	5		Valida i dati inseriti
	6		Salva la issue e aggiorna la lista
	7		
	8		
Extension A: Dati mancanti	5a		Segnala errore sui campi
	6a	Corregge i campi	Torna al passo 5

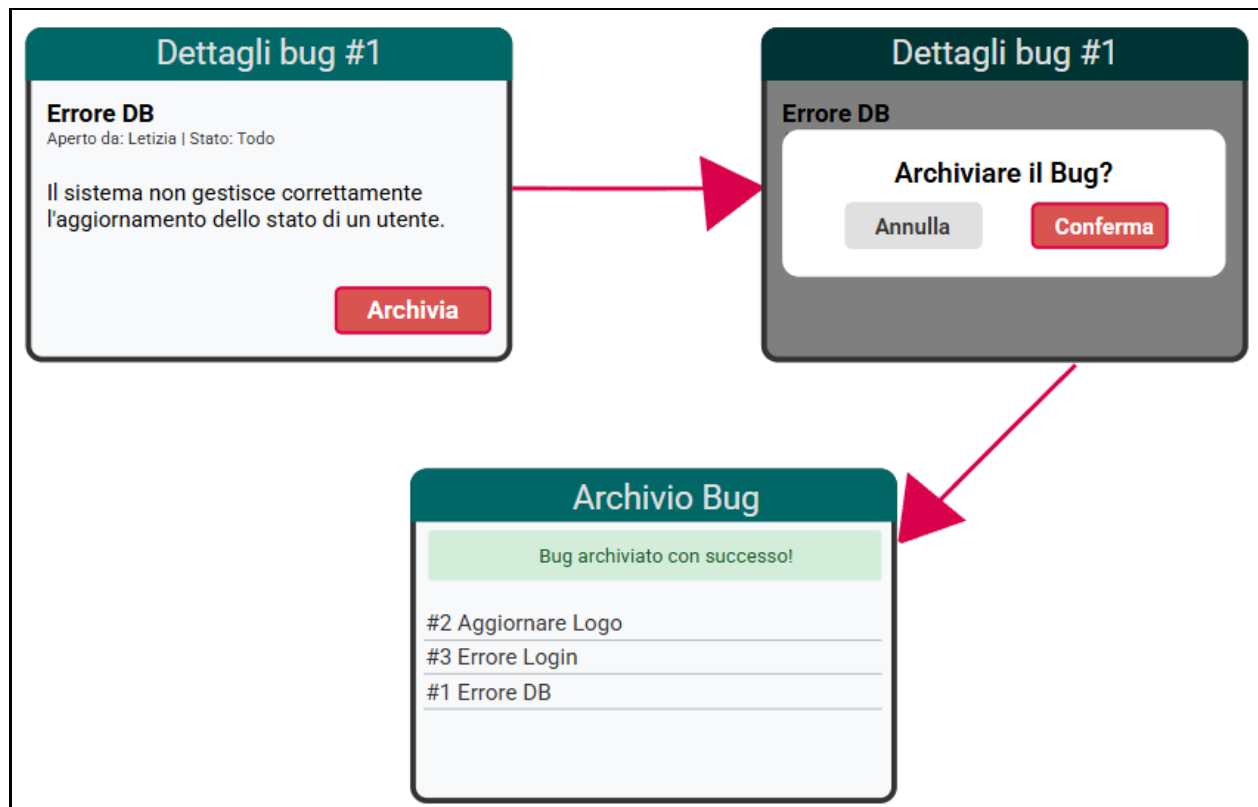
5.2 UC2: Archiviazione di un Bug

USE CASE #2	Archiviazione di una Issue		
Goal in Context	Permettere a un amministratore di archiviare una issue, rimuovendola dalle liste principali.		
Preconditions	L'utente deve essere autenticato come Amministratore e la issue deve esistere.		
Success End Condition	La issue viene rimossa dalle viste principali e spostata nella pagina dedicata.		
Failed End Condition	Lo stato della issue rimane inalterato (es. operazione annullata).		
Primary Actor	Amministratore		
Trigger	L'amministratore seleziona l'opzione "Archivia" su una issue.		
Main Scenario	Step n.	Actor 1 (Admin)	System
	1		Chiede conferma dell'archiviazione tramite pop-up
	2	Conferma l'archiviazione	
	3		Modifica lo stato in "Archiviato"
	4		Rimuove il bug dalle liste principali
	5		Lo rende visibile nella pagina dedicata
	6		Mostra messaggio di successo
	7		
Extension A: Annullamento	2a	Annulla l'operazione	
	3a		Chiude la finestra di conferma e annulla l'operazione

5.3 Mock-Up UC1: Segnalazione di una Issue



5.4 Mock-Up UC2: Archiviazione di un Bug



6 Architettura Proposta

L'architettura del sistema BugBoard26 è stata progettata seguendo il paradigma Client-Server, al fine di realizzare un sistema distribuito composto da due macro-componenti completamente indipendenti: il Back-end e il Front-end.

6.1 Back-end (Server)

Il Back-end è stato progettato per operare in totale autonomia rispetto all'interfaccia utente. Esso si fa carico dell'intera logica di business e della persistenza dei dati, garantendo la gestione centralizzata dello stato del sistema.

La struttura del Back-end è organizzata in componenti Model, Repository, Service e Controller, ciascuno caratterizzato da responsabilità specifiche.

I Controller gestiscono le richieste HTTP, i Service concentrano la logica applicativa, i Repository gestiscono l'accesso ai dati persistenti e i Model rappresentano le principali entità del dominio.

Per consentire la comunicazione con il Front-end, il Back-end espone un set di interfacce di programmazione via rete (API REST), permettendo così al sistema di essere fruibile da qualsiasi client autorizzato. Inoltre, per garantire la massima portabilità e facilitare il futuro deployment su infrastrutture Cloud, il servizio è predisposto per essere containerizzato utilizzando Docker.

6.2 Front-end (Client)

Il Front-end rappresenta l'interfaccia attraverso cui gli utenti interagiscono con la piattaforma. Nel rispetto dei vincoli architetturali, il Front-end è privo di logica di business complessa e comunica con il Back-end esclusivamente tramite chiamate di rete alle API esposte. Internamente, il client è stato strutturato seguendo il pattern architetturale MVC (Model-View-Controller). Quest'ultima garantisce un basso accoppiamento e assicura che il client possa essere interamente sostituito o modificato senza richiedere alcun intervento sul codice sorgente del Back-end.

6.3 Criteri di Design e Astrazione

La scelta di adottare un'architettura distribuita Client-Server è motivata dai seguenti criteri di design:

- **Separazione delle responsabilità:** Mantenendo la logica di business isolata dalla presentazione visiva e strutturando il codice per livelli, si ottiene un sistema altamente coeso ed a basso accoppiamento.
- **Astrazione e Riutilizzabilità:** Le API del Back-end fungono da strato di astrazione. Questo favorisce il riutilizzo del codice lato server qualora, in futuro, si decidesse di sviluppare ulteriori client.
- **Scalabilità:** La separazione tra Front-end e Back-end consente di evolvere e mantenere i due componenti in maniera indipendente, facilitando eventuali modifiche future e l'introduzione di ulteriori client.

7 Scelte Tecnologiche

La selezione delle tecnologie è ricaduta su strumenti moderni, robusti e conformi al paradigma Object-Oriented. Questa decisione è motivata dalla necessità di realizzare un'architettura facilmente manutenibile nel tempo: l'approccio orientato agli oggetti ci permette infatti di sfruttare i principi di incapsulamento e astrazione, riducendo l'accoppiamento tra i moduli e facilitando l'eventuale introduzione di nuove funzionalità. Inoltre, tale paradigma risulta fondamentale per massimizzare la modularità e il riuso del codice, oltre a garantire una corrispondenza diretta e coerente tra i modelli di design e l'implementazione effettiva del sistema.

7.1 Tecnologie Back-end

Per la realizzazione del Back-end, si è scelto di adottare il linguaggio **JAVA**. L'adozione di tale tecnologia è motivata dalla sua natura fortemente tipizzata e dal suo paradigma Object-Oriented, caratteristiche essenziali per garantire robustezza, stabilità e sicurezza del codice. Java inoltre è stato affiancato al framework **Spring Boot**, selezionato per la sua capacità di agevolare l'implementazione delle API REST. Tale framework implementa in modo eccellente il pattern della *Dependency Injection*, favorendo un forte disaccoppiamento dei componenti software.

La persistenza è stata implementata mediante **Spring Data JPA**, che permette di interagire con PostgreSQL attraverso il mapping tra le entità Java e le tabelle relazionali. **PostgreSQL** è un DBMS relazionale open source che garantisce affidabilità e integrità referenziale delle principali entità di dominio del sistema, quali utenti, issue e allegati. La logica applicativa è gestita interamente all'interno del codice Back-end, evitando di delegare regole di business a servizi *MBaaS* esterni.

Per la gestione dell'autenticazione e dell'autorizzazione è stato utilizzato **Spring Security**, integrato con un meccanismo di autenticazione basato su **JWT**. In questo modo il Back-end può verificare il token associato alle richieste e applicare controlli di accesso basati sul ruolo dell'utente.

7.2 Tecnologie Front-end

Il Front-end è stato sviluppato utilizzando **JavaFX**. È una piattaforma grafica per lo sviluppo di applicazioni desktop in ambiente Java è stata scelta per la sua capacità di creare interfacce desktop (GUI) fluide, reattive e nativamente multi-piattaforma.

L'uso di *JavaFX* congiunto al linguaggio di markup **FXML** ha permesso di strutturare il front-end secondo il pattern MVC, isolando completamente la progettazione visiva delle schermate (*View*) dalla logica di gestione degli eventi (*Controller*). La comunicazione con le API del Back-end è stata invece affidata a un livello **Client** dedicato (Sfruttando **HttpClient**), supportato da apposite classi Helper. Per la gestione dello scambio di dati in formato **JSON** tra Front-end e Back-end viene utilizzata la libreria **Jackson**. Essa permette di effettuare la *serializzazione* degli oggetti Java in JSON e la *deserializzazione* dei dati JSON negli oggetti Java corrispondenti. In questo modo, il client può convertire automaticamente le risposte ricevute dalle API REST in oggetti Java e serializzare i dati da inviare al Back-end, semplificando la gestione della comunicazione tra i due componenti.

7.3 Strumenti di Sviluppo e Modellazione

Per le fasi di analisi e progettazione orientata agli oggetti è stato impiegato il tool CASE **Visual Paradigm**, che ha permesso la realizzazione e l'esportazione formale dei diagrammi UML. Per quanto riguarda il processo di sviluppo e la collaborazione in team, la gestione del codice sorgente è stata interamente affidata a **Git**, utilizzando **GitHub** come piattaforma di hosting per il repository. L'adozione di quest'ultimo sistema, non è stata relegata a un semplice strumento di backup, ma è stata fondamentale per coordinare il lavoro parallelo dei membri del gruppo. Attraverso l'utilizzo strutturato dei branch, si è garantita l'integrità del codice sul ramo principale, permettendo lo sviluppo isolato delle singole feature dell'applicativo.

Per assicurare il rispetto degli standard di qualità di codice e dei principi di *clean code*, è stato impiegato **SonarQube**. Questo strumento di analisi continua si è rivelato fondamentale per rilevare *code smells*, bug e vulnerabilità.